飞书解决方案案例 · 求职作品

把战队运营搬进飞书

多维表格 + 机器人 + 面板：把任务、提案、物料、合同四条链路，收进一个成员每天都在用的入口。

案例对象：辽宁工程技术大学「奇点蓝图」RoboMaster 战队　时间：2026.08 – 至今　角色：个人主导

这套系统的起点不是技术，而是一个具体的管理困境：16 人、四个技术方向、没有行政约束力，任务靠群消息、库存靠 Excel、赞助合同靠记忆，队长成了人肉路由器。我的做法是不新开系统——把飞书多维表格当作唯一数据源，向上接表单、自动化、机器人和一块面板，让流程自己跑。

16	6	3	3
条任务 / 提案记录	条自动化流程	个业务机器人	个战队群接入

> 边界说明：任务提醒与面板方案参考 RoboMaster BBS 1884257，做本地化改造与扩展；BOM 机器人原型来自开源项目，我接管并改造了后端判断逻辑；采购、赞助两个机器人按同一模板新增。案例是学生战队场景，不是制造业客户交付；文中数字来自飞书多维表格与本地运行日志，未做估算。

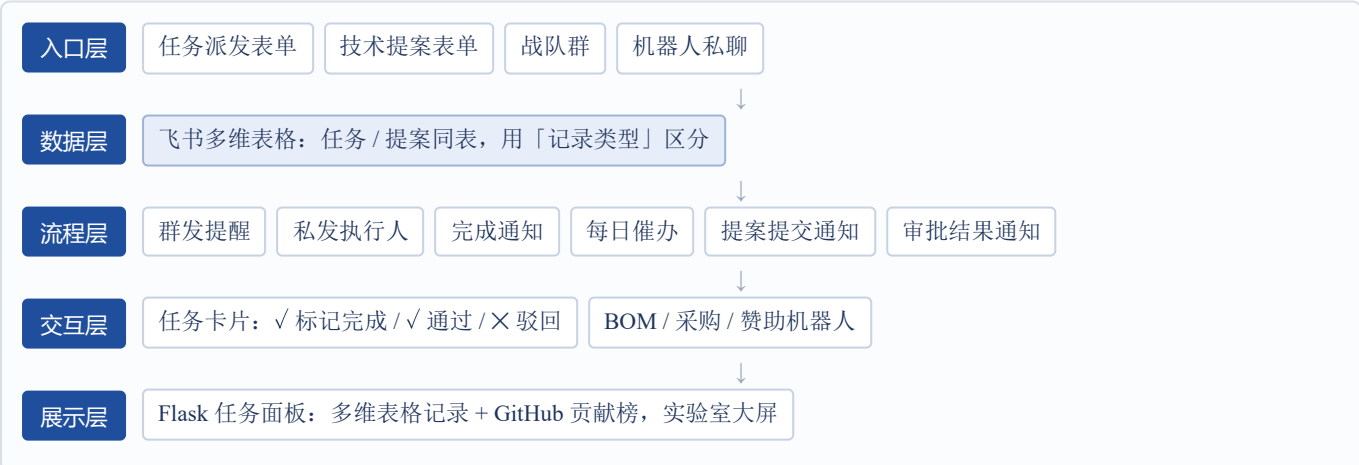
一、场景与约束

学生组织没有行政约束力，成员分布在机械、电控、软件、视觉四个方向，作息和投入度差异很大。三件事最容易掉地上：

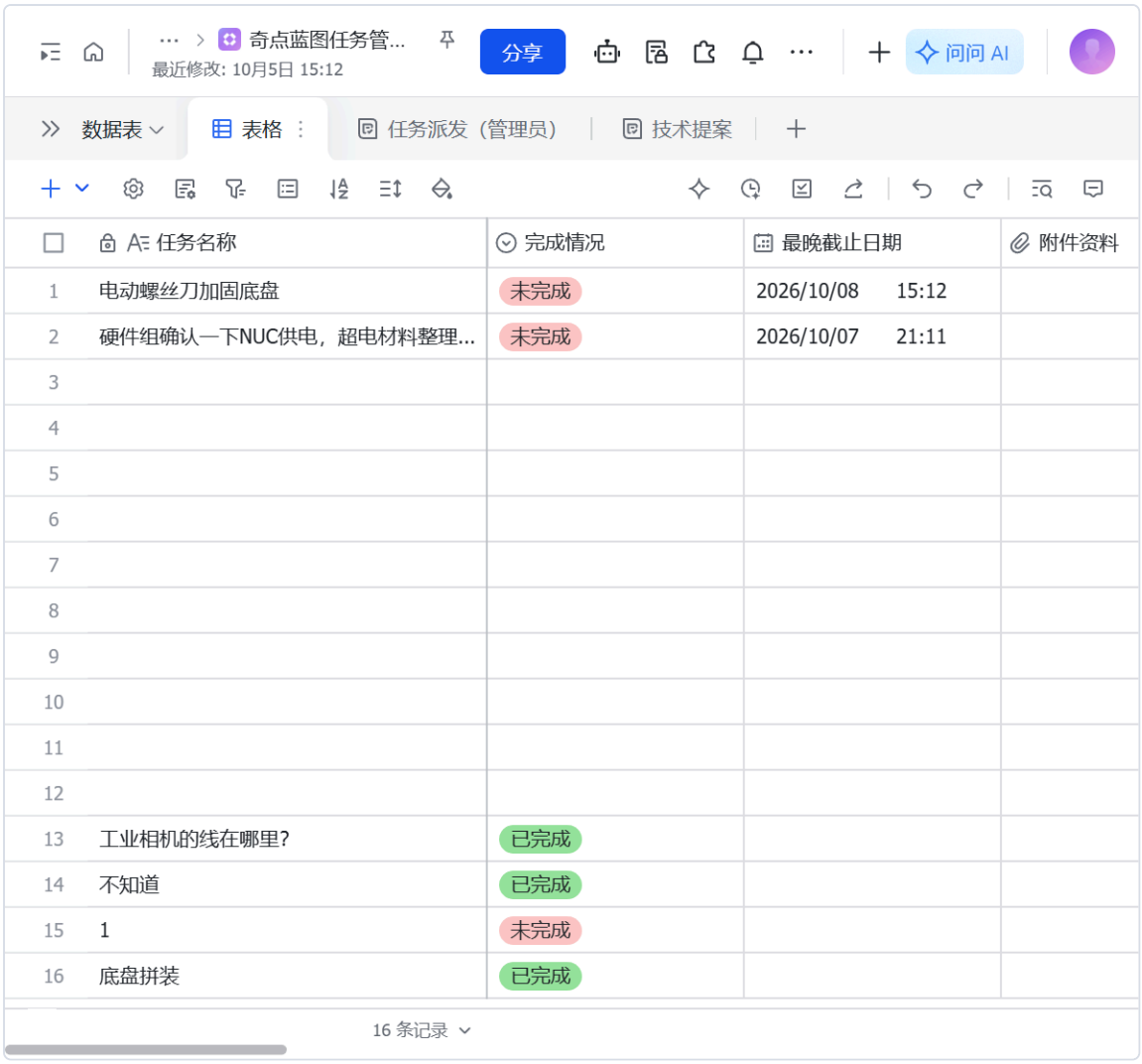
- 任务派了没人跟：群里 @ 一下就算派发，截止日期靠记忆，催办靠队长逐个私聊；
- 物料账对不上：元器件用 Excel 台账，查一颗料要翻表、问人，采购过的件重复买；
- 信息不回流：队员有技术想法，但没有低成本的上报入口；提案和任务混在一起，无法区分。

约束决定了方案形态：不新增系统、不改变成员习惯。成员已经在飞书里，所以入口必须是飞书；队长只有一个人，所以流程必须自动跑。

二、方案总览：四层结构



四层里，数据层只有一个源：所有入口写进同一张多维表格，所有展示和通知都从这张表读。这样新增一条链路时，不需要再建一套数据。



飞书多维表格「奇点蓝图任务管理后台」：任务派发与技术提案共用一张表，底部可见 16 条记录；顶部是两张表单入口。

三、四条链路

A. 任务派发（自上而下）

填任务派发表单 → 新增记录 → 群发卡片 → 私发执行人 → √ 标记完成 → 通知群与发布人 → 每日 09:00 催办

关键设计：卡片按钮用多维表格的 `setRecord` 直接改单选值，不引入外部服务，点击即闭环；催办只发执行人、不发群，避免公开施压；截止日期、优先级、附件留在记录里，群消息只承担提醒，不承担存档。

> 运行证据：10-01 17:06 发出「📌 你有一个新任务」；10-02 至 10-05 每天 09:00 连续发出「⏰ 任务未完成提醒」；09-29 测试任务点击「√ 标记完成」后，群与发布人收到「✅ 任务已完成」。

B. 技术提案（自下而上）

队员填提案表单 → 记录类型 = 技术提案 → 审核状态 = 待审核 → 私信队长 → √ 通过 / ✕ 驳回 → 通知提案人 → 手动转任务

关键设计：同表加字段、不新建表——新增「记录类型」「审核状态」两个字段，避免多表维护成本，历史记录用 `!t ||` 兜底；审批只提醒、不自动建任务，避免把没想清楚的想法直接变成排期；提案不上大屏，待评审信息不适合公屏展示；通知目标写成独立配置项，后续从「私信队长」改成「发管理群」只动一处。

> 运行证据：09-29 17:19 收到「💡 有新的技术提案待审批」；一条提案已完成审批（审核状态 = 已通过）。

C. BOM 库存机器人

群里上传 BOM → 加入库存 / 摘取物料 → 对比库存 → 生成精简 BOM → 入库 + 同步多维表格 → 查询 / 历史 / 删除

- 长连接：用 lark-oapi 走纯出站 WebSocket，不需要公网回调；Flask Webhook 仅作备选；
- 会话状态机：5 个状态覆盖「文件已上传 → 选模式 → 确认 → 完成」，避免多步操作串线；
- **AI** 兜底：OpenRouter 做去重与命名标准化，失败时降级到本地规则，不阻断主流程；
- 可回滚：删除按批次回滚，避免误删整库；库存同步到多维表格，对话与表格看同一份数据。

我的改造：接管开源项目的后端判断逻辑，补上「成员提想法 → 队长审批 → 结果回访」的反馈闭环；把这套「长连接 + 状态机 + 表格同步」的模板抽象出来，复制到采购与赞助。

D. 采购与赞助合同

采购机器人

申请 → 审批 → 到货验收，机器人做状态查询与提醒。代码已完成，尚未上线。

赞助合同机器人

「草稿 / 生效中 / 已完成 / 已终止」状态机，多步引导录入，同步多维表格；09-24 完成长连接联机验证。

四、面板与部署

任务面板（**Flask**）：`/api/feishu` 分页拉取多维表格全量记录（每页 500 条，最多 10 页）；`/api/github` 汇总组织仓库贡献榜与动态，10 分钟文件缓存，节省 API 配额；前端是赛博风大屏，挂在实验室显示器上，任务总数、待办排行、DDL 倒计时实时刷新。

进程守护：Windows 侧用 `run_all.py` 一个进程管三个机器人：子进程崩溃自动重启、单实例锁防止同一应用两条长连接互相抢消息；上云方案为阿里云 ECS 学生机 + 三个 systemd 服务（面板 / BOM / 赞助），5010 高位端口绕开备案，代码零改动，密钥单独传输。

> 运行方式：任务后台的 6 条自动化在飞书云端常驻；三个业务机器人走飞书长连接，已验证可正常连上并按需启动。

五、结果与证据

维度	数字 / 状态	证据
数据	16 条记录、10 个字段、2 张表单	多维表格「奇点蓝图任务管理后台」
流程	6 条自动化全部启用	群发 / 私发 / 完成通知 / 每日催办 / 提案通知 / 审批结果
交互	3 个业务机器人	BOM、采购、赞助；任务后台另有独立应用
覆盖	3 个战队群、16 人	2027 赛季步兵、哨兵、硬件组
运行	连续 4 天每日 09:00 催办	10-02 – 10-05 私信卡片
闭环	任务完成、提案审批均可回写	09-29 两条卡片记录

六、如果换到制造业：这套东西怎么迁移

制造业场景	本案例对应物	可直接复用的部分
研发	任务派发 + 技术提案双通道	同表双记录类型、审批卡片、结果回写
生产	缺料上报、线边库查询	BOM 库存机器人的查询 / 报缺 / 回滚
供应	BOM 解析、采购申请	BOM 机器人 + 采购机器人的状态机
销售	赞助合同跟踪	合同状态机（可换为客户合同 / 回款）
服务	群内答疑	机器人问答 + 知识库沉淀

换行业时不需要重写系统，需要重写的是字段、审批规则和话术——这正是解决方案岗要做的翻译工作。

七、复盘

做对了

- 先跑通一条链路（任务派发），再复制到提案、库存、合同，避免一次性设计过大；
- 入口放在成员已经在用的飞书里，推广成本接近零；
- 同表加字段而不是新建表，自动化和面板只改一处；
- 卡片按钮直接改记录，不引入额外服务，点击即闭环。

不足

- 使用量小：16 条记录里含测试数据，真实任务只有个位数；
- BOM 主数据还可进一步收敛到多维表格，当前是本地 JSON + 云表格同步；
- 机器人按需启动，缺少健康检查与告警；
- 权限只做到名单级（删除 / 管理员），没有按角色细分。

下一步：把 BOM 主数据迁到多维表格、加健康检查与告警、把「任务按时完成率 / 提案通过率」做成面板指标。

八、未来展望：从「人做 Router」到 Super-agent

现在这套系统的瓶颈不在数据，而在队长仍然是 **Router**：任务、提案、缺料、合同都要人先看见、再判断、再分派；三个机器人是命令式 Worker，不会主动找活；知识沉淀靠事后手写。

DeepSeek Harness 团队 2026-10-31 开源的 **Super-agent** 架构，给了一个可参照的下一步：Router 常驻接收群聊 / 表单 / 邮件 / CI 事件，两级过滤（程序规则 + 模型研判）决定接不接；接了就建 Ticket，交给上下文隔离的 Worker 并行执行，结果从 Channel 回到原对话。官方强调这是研究原型（`dsh-superagent-prototype`，MIT），不是正式产品。

对应到这套系统，可以分四步走：

1. 事件入口不换：现有 3 个群、表单、多维表格继续用；在群消息 / 表单提交上增加 Trigger 与两级过滤，先挡掉闲聊，再判断是否建单。
2. **Ticket** 化：任务、提案、缺料、考核、合同统一成 Ticket，多维表格做状态机；Worker 独立上下文执行，结果回原群 / 原表单。
3. 主动执行：缺料讨论 → 自动查 BOM → 生成补料建议；技术提案 → 预审 → 转任务；考核临期 → 生成行动项；合同到期 → 提醒。先选一条链路（缺料）验证，再扩展。
4. 知识运营：每个 Ticket 关闭时自动生成复盘卡（问题 / 动作 / 结果 / 可复用点），回流案例库，对应「最佳方案、案例、打单心法」的沉淀。

> 边界：以上是展望，不是已实现；Super-agent 是研究原型，官方明确不是正式产品；现有系统先保持「规则 + 自动化 + 命令式机器人」，不为了新架构重写。

一句话：把「人盯会话」换成「系统管调度、人管目标与验收」——这和这套系统一开始的判断一致。

九、代码结构

```
bom-manager-main/
├─ bom_inventory/        BOM 解析、库存 CRUD、会话状态机、飞书长连接机器人
├─ purchase_manager/     采购申请机器人（未上线）
├─ sponsor_contracts/    赞助合同机器人（状态机 + 多维表格同步）
├─ task_dashboard/       Flask 面板（多维表格记录 + GitHub 贡献榜）
└─ _launcher/            Windows 进程总管（单实例锁 + 自动重启）
```

本案例为个人求职作品，数据来自飞书多维表格与本地运行日志；涉及队友姓名的地方已省略。

十、参考

任务提醒与面板方案参考：RoboMaster BBS《连接飞书的一体化任务提醒及面板-任务管理系统》（原文 CC BY-NC-SA 4.0）。本案例在其基础上做战队场景的本地化改造与扩展，不用于商业用途。